x86-64 Assembly Programming Problem Set

Practice Programming problems

Instructions

- Every problem asks for a **complete x86-64 program** (NASM/SASM syntax). Sample runs show the expected output;
- Each program must use at least one functional x86-64 register. Unless told otherwise, store results in the named memory locations *and* display them.
- Assume the SASM I/O macros used in lecture (see Appendix A). Assume well-formed input unless the problem asks for validation.
- Problems are grouped by topic. Section 5 (searching) is the focus of recent exams; Section 9 contains a mock exam assembled from the problem bank. Hints, including binary search pseudocode, are in Appendix B.

Contents

1 Data Transfer and Addressing **2**

2 Arithmetic **2**

3 Logic and Bit Manipulation **3**

4 Loops and Arrays **4**

5 Searching (Binary Search) **5**

6 Strings **6**

7 Functions and the Stack **8**

8 Number and Input-Validation Programs **9**

9 Mock Exam **11**

A SASM I/O Macro Reference **12**

B Hints **13**

1 Data Transfer and Addressing

1.1 Swap. *(10 pts)*

Declare two 64-bit variables `A dq 0x1111_2222_3333_4444` and `B dq 0x5555_6666_7777_8888`. Swap their contents using only `MOV` and registers (no `XCHG`). Display both values in hex before and after the swap.

1.2 Table lookup with LEA. *(10 pts)*

Declare `TBL dw 0x0A0B, 0x1C1D, 0x2E2F, 0x3031, 0x4243`. Use `LEA` to put the address of `TBL` in a register, then load the **fourth** word into `AX` using an offset from that register. Display `AX` in hex.

```
AX = 3031
```

1.3 Copy a string. *(10 pts)*

Copy the null-terminated string `SRC db "assembly", 0` into a buffer `DST` byte by byte using two pointer registers (including the terminating 0). Display both strings.

2 Arithmetic

2.1 Expression. *(10 pts)*

Declare `A dd 12`, `B dd 8`, `C dd 5`, `D dd 30`, and `RESULT dq 0`. Compute $(A + B) \times C - D$, store it in `RESULT`, and display it in decimal.

```
Result: 70
```

2.2 Quotient and remainder. *(10 pts)*

Read two signed integers and display their quotient and remainder. If the divisor is 0, display an error message instead of dividing.

```
Enter dividend: -17
Enter divisor: 5
Quotient: -3
Remainder: -2
```

2.3 Average of three. *(10 pts)*

Read three integers and display their average as a quotient and a remainder.

```
Enter three numbers: 6 7 9
Average: 7 remainder 1
```

2.4 Sum of the first n integers. *(10 pts)*

Read a positive integer n and compute $\frac{n(n+1)}{2}$. The program must support result values up to 2^{32} and beyond, so use 64-bit registers.

```
Enter n: 100000
```

```
Sum: 5000050000
```

2.5 Seconds to hours, minutes, seconds. *(10 pts)*
Read a number of seconds and display it as hours, minutes, and seconds using `DIV`.

```
Enter seconds: 3725
1 hour(s) 2 minute(s) 5 second(s)
```

2.6 Temperature conversion. *(10 pts)*
Read a Celsius temperature C (integer) and display $F = \frac{9C}{5} + 32$ as a whole part and a remainder (the remainder is in fifths). Negative inputs must work (use signed instructions).

```
Enter Celsius: 37
Fahrenheit: 98 remainder 3/5
```

3 Logic and Bit Manipulation

3.1 Uppercase conversion. *(10 pts)*
A string is stored in `STR db "hello world", 0`. Convert every lowercase letter to uppercase using `AND` (leave other characters unchanged), store the result in `OUT`, and display both strings.

3.2 Even or odd. *(10 pts)*
Read an integer and use `BT` (or `AND` with a mask) to decide whether it is even or odd. Display `Even` or `Odd`.

3.3 Population count. *(15 pts)*
Count the 1-bits in `NUM dq 0x0F0F_F0F0_1234_5678` using `BT` in a loop. Display the count in decimal.

```
Number of 1-bits: 29
```

3.4 Set, clear, toggle. *(10 pts)*
Given `FLAGS db 0x5A`, use `OR`, `AND`, and `XOR` (in that order) to (a) set bit 0, (b) clear bit 3, (c) toggle bit 7. Display the value in hex after each step.

```
After set bit 0:    5B
After clear bit 3:  53
After toggle bit 7: D3
```

3.5 Power of two. *(10 pts)*
Read a positive integer n and decide whether it is a power of two using the identity $n \,\&\, (n-1) = 0$. Display `Yes` or `No`. Test with 64, 96, and 1.

4 Loops and Arrays

4.1 Count a null-terminated list. *(10 pts)*
Count the elements of `ALPHA db 0x12, 0x34, 0x56, 0x99, 0x00` (terminated by 0). Store the count in `COUNT` and display it.

4.2 Maximum and minimum. *(10 pts)*
Given `ARR db 5, -3, 12, 7, -9, 0, 21, -1` (signed bytes, 8 elements), find and display the maximum and minimum.

```
Max: 21
Min: -9
```

4.3 Sign counts. *(10 pts)*
For the same array, count how many elements are negative and how many are positive (zero counts as neither). Display both counts.

```
Negative: 3
Positive: 4
```

4.4 Array statistics. *(15 pts)*
Given `ARR dd 14, 7, 92, 3, 55, 28, 61, 9`, compute and display the sum, the average (quotient and remainder), the maximum, and the minimum.

```
Sum: 269
Average: 33 remainder 5
Max: 92
Min: 3
```

4.5 Linear search. *(10 pts)*
Given `ARR dd 14, 7, 92, 3, 55` (5 elements), read a key and display the 0-based index of its first occurrence, or `Not found`.

```
Enter key: 3
Found at index 3
```

4.6 Reverse in place. *(10 pts)*
Reverse `ARR dd 1, 2, 3, 4, 5, 6, 7` in place using two index registers that move toward each other (no second array). Display the array before and after.

```
Before: 1 2 3 4 5 6 7
After:  7 6 5 4 3 2 1
```

4.7 Bubble sort. *(15 pts)*
Sort `ARR dd 29, 4, 17, 8, 52, 1, 33, 12` in ascending order using bubble sort (nested loops; use `LOOP` for at least one of them). Display the array before and after.

```
Before: 29 4 17 8 52 1 33 12
After:  1 4 8 12 17 29 33 52
```

5 Searching (Binary Search)

Unless stated otherwise, use this sorted array of 12 distinct 32-bit values:

```
ARR   dd 4, 9, 13, 22, 27, 35, 41, 56, 63, 70, 84, 99
COUNT dd 12
```

Indices are 0-based. You may use `RSI`, `RDI`, `RBX`, etc. for the low, high, and mid indices. Your program must run in $O(\log n)$ steps: do not scan the array linearly.

5.1 Basic binary search. *(15 pts)*

Read a key and search `ARR` using iterative binary search. Display the 0-based index where the key is found, or `Not found`.

```
Enter key: 63
Found at index 8

Enter key: 50
Not found
```

5.2 Binary search with probe count. *(15 pts)*

Extend Problem 5.1: count how many array elements you compared with the key (each loop iteration is one probe) and display this count in both the found and not-found cases.

```
Enter key: 70
Found at index 9 after 4 probe(s)

Enter key: 50
Not found after 4 probe(s)

Enter key: 35
Found at index 5 after 1 probe(s)
```

5.3 Binary search as a function. *(20 pts)*

Write a function `BSEARCH` that receives, on the stack, (1) the address of the array, (2) the number of elements, and (3) the key. It returns the index in `RAX`, or -1 if not found. Clean up the parameters with `ret 24`. From `CMAIN`, call it for the keys 56, 99, and 5 and display each result.

```
56 -> 7
99 -> 11
5  -> -1
```

5.4 Descending order. *(15 pts)*

The array `DESC db 90, 77, 64, 51, 38, 25, 12` (7 unsigned bytes) is sorted in **descend-**

ing order. Modify binary search accordingly and search for a key read from the user. Display the index or `Not found`.

```
Enter key: 38
Found at index 4

Enter key: 100
Not found
```

5.5 Sort, then search. *(20 pts)*

Given the unsorted array `ARR dd 29, 4, 17, 8, 52, 1, 33, 12`, first sort it in ascending order (any sort), display the sorted array, then read a key and binary-search the sorted array.

```
Sorted: 1 4 8 12 17 29 33 52
Enter key: 29
Found at index 5
```

5.6 Binary search on characters. *(15 pts)*

The string `LETTERS db "acdfhkmprtwz", 0` is sorted alphabetically. Read one character and binary-search for it (the length is found first by scanning for the terminator, or by computing it once). Display its index or `Not found`.

```
Enter character: k
Found at index 5

Enter character: b
Not found
```

5.7 Recursive binary search (challenge). *(25 pts)*

Write `BSEARCH` recursively: each call receives the array address, `low`, `high`, and the key on the stack, and either returns the index in `RAX` or calls itself on the left or right half. Use the same test keys as Problem 5.3.

6 Strings

6.1 DNA string: replace T with U. *(10 pts)*

A DNA string is stored in memory location `STRVAR` and terminated with 0. Assume the DNA string is at most 40 characters long. Write an x86-64 program to replace every occurrence of the character "T" with the character "U" and store it in memory location `OUTPUT`. Also, count the length of the DNA string and keep it in the memory location `DNALEN`. Display both the size and the modified DNA string on the output screen.

Place between 30–40 DNA characters of your choice for your DNA string declaration. You are required to use at least one functional x86-64 register in the program. Write your name and section on the 1st line of the code as a comment.

Sample declaration:

```
STRVAR  db "ACGTACGTCCCTTT", 0
```

```
OUTPUT  times ?? db 0     ; replace ?? with the appropriate number
DNALEN  db 0
```

(*The original uses // for the comment; NASM comments start with ;.*)

6.2 Complementary strand. *(10 pts)*
Declare a DNA string of 30–40 characters (terminated by 0) in `STRVAR`. Produce its complement in `COMP` by replacing A↔T and C↔G. Display the original, the complement, and the length. Use a `switch`-style structure (compare and jump).

6.3 GC content. *(10 pts)* For a DNA string of 30–40 characters, count the G's and C's and display the count and the percentage of GC content (integer division; also show the remainder).

6.4 Character frequency. *(10 pts)*
Given `TEXT db "mississippi", 0`, read a character and display how many times it occurs.

```
Enter character: s
Count: 4
```

6.5 Vowel counter. *(10 pts)*
A lowercase sentence of at most 40 characters is stored in `SENT` (terminated by 0). Count the vowels (a, e, i, o, u) and display the count.

```
Sentence: assembly is fun
Vowels: 4
```

6.6 Word count. *(10 pts)*
Count the words in `SENT db "the quick brown fox", 0`. Assume words are separated by exactly one space and there are no leading or trailing spaces.

```
Words: 4
```

6.7 Caesar cipher. *(15 pts)*
Encrypt a lowercase word by shifting each letter forward by 3 positions, wrapping around from `z` to `a`. Store the result in `CIPHER` and display both strings.

```
Plain:  hello
Cipher: khoor

Plain:  xyz
Cipher: abc
```

6.8 Palindrome checker. *(15 pts)*
A word of at most 20 lowercase letters is stored in `WORD` (terminated by 0). Determine

whether it is a palindrome (use two pointers or the stack) and display `Palindrome` or `Not a palindrome`. Test with "racecar" and "assembly".

6.9 Reverse a string using the stack. *(15 pts)*
Reverse `MSG db "assembly", 0` by pushing each character onto the stack and popping them back into `REV`. Display the original and reversed strings.

```
Original: assembly
Reversed: ylbmessa
```

7 Functions and the Stack

For all problems here, parameters are pushed by the caller, accessed through `RBP`-relative addressing, and removed by the callee with `ret n`.

7.1 ADDTWO. *(10 pts)*
Write a function `ADDTWO` that receives two 64-bit values on the stack and returns their sum in `RAX`. Call it with 25 and 17 and display the result.

7.2 MAXOF. *(10 pts)* Write a function `MAXOF` that receives two signed 64-bit values and returns the larger in `RAX`. Test with $(-5, 3)$ and $(10, 10)$.

7.3 SQUARE. *(10 pts)*
Write a function `SQUARE` that receives one value and returns its square in `RAX` (64-bit). Display the squares of 7, 1000, and 65535.

```
7 -> 49
1000 -> 1000000
65535 -> 4294836225
```

7.4 Factorial. *(15 pts)*
Write a function `FACT` that returns $n!$ in `RAX` using a loop. Test with $n = 10$ and $n = 20$.

```
10! = 3628800
20! = 2432902008176640000
```

7.5 Power. *(15 pts)*
Write a function `POWER` that receives a base and a non-negative exponent and returns base^{exp} in `RAX`. Test with $(3, 13)$ and $(2, 40)$.

```
3^13 = 1594323
2^40 = 1099511627776
```

7.6 GCD function. *(15 pts)*
Write a function `GCD` that returns the greatest common divisor of two positive integers using the Euclidean algorithm (repeated `DIV`). Display `GCD(48, 18)` and `GCD(17, 5)`.

```
GCD(48, 18) = 6
GCD(17, 5) = 1
```

7.7 Array sum function. *(15 pts)*
Write a function `SUMARR` that receives the address of a 32-bit array and its length, and returns the sum in `RAX`. Use it on `ARR dd 14, 7, 92, 3, 55, 28, 61, 9`.

8 Number and Input-Validation Programs

8.1 Square: perimeter and area. *(10 pts)*
Write an x86-64 assembly program to compute the perimeter and area of a square. The side of the square is obtained from the input prompt. Output the required result. There should be an error check for negative input. The program should support result values up to 2^{32}.

You are required to use at least one functional x86-64 register in the program. Write your name and section on the 1st line of the code as a comment.

```
***Example #1:
Enter length: 20
Perimeter: 80
Area: 400

***Example #2:
Enter length: -20
Error: Negative input not allowed, please try again.
Enter length: 2
Perimeter: 8
Area: 4
```

8.2 Rectangle with validation. *(10 pts)*
Read the length and width of a rectangle. Reject negative values and zero with an error message and prompt again. Display the perimeter and area. Support results up to 2^{32}.

8.3 Sum of digits. *(10 pts)*
Read a positive integer (up to 10 digits) and display the sum of its digits using repeated division by 10.

```
Enter number: 98765
Sum of digits: 35
```

8.4 Fibonacci. *(10 pts)*
Read n ($1 \leq n \leq 40$) and display the first n Fibonacci numbers using `LOOP`.

```
Enter n: 8
1 1 2 3 5 8 13 21
```

8.5 Prime test. *(15 pts)*

Read an integer $n \geq 2$ and determine whether it is prime by testing divisors from 2 up to $n - 1$ (or up to $\sqrt{n}$ for extra credit). Display `Prime` or `Composite`.

8.6 Decimal to binary. *(15 pts)*

Read a positive integer and display its binary representation without leading zeros (use `BT` or shifts, or repeated division by 2 with the stack).

```
Enter number: 37
Binary: 100101
```

9 Mock Exam

Suggested time: 90 minutes. Solve without notes first, then check against the sample runs. Total: 50 points.

9.1 Binary search. *(20 pts)*
Using `ARR dd 4, 9, 13, 22, 27, 35, 41, 56, 63, 70, 84, 99` (12 elements, sorted ascending), read a key and use **binary search** to find it. Display the index of the key, or `Not found`, and the number of probes made. A linear search will receive no credit.

9.2 String processing. *(15 pts)*
A string of at most 40 lowercase characters is stored in `STR` (terminated by 0). Write a program that (a) counts the number of vowels, (b) converts the entire string to uppercase and stores it in `OUTPUT`, and (c) displays the length, the vowel count, and the converted string.

9.3 Function with stack parameters. *(15 pts)* Write a function `POWER` that takes a base and an exponent on the stack and returns the result in `RAX`. Use it to compute and display 3^{13} and 2^{40}.

A SASM I/O Macro Reference

The following macros are assumed (as used in lecture). Adjust to your course environment as needed.

Macro	Purpose
`PRINT_STRING msg`	Print a null-terminated string or literal
`PRINT_CHAR ch`	Print one character
`PRINT_DEC size, reg/mem`	Print signed decimal (size = 1, 2, 4, 8 bytes)
`PRINT_UDEC size, reg/mem`	Print unsigned decimal
`PRINT_HEX size, reg/mem`	Print hexadecimal
`GET_DEC size, reg/mem`	Read signed decimal into register/memory
`GET_UDEC size, reg/mem`	Read unsigned decimal
`GET_CHAR reg/mem`	Read one character
`NEWLINE`	Print a line break

Program skeleton

```
; Name, Section
%include "io64.inc"

section .data
    ; declarations here

section .text
global CMAIN
CMAIN:
    mov rbp, rsp; for correct debugging
    ; your code here
    xor rax, rax
    ret
```

B Hints

Binary search pseudocode (Section 5)

```
low  = 0
high = n - 1
while (low <= high)
    mid = (low + high) / 2
    if (ARR[mid] == key)  found at mid
    if (ARR[mid] <  key)  low  = mid + 1
    else                  high = mid - 1
not found
```

- Keep `low`, `high`, and `mid` in 64-bit registers. Compute `mid` with `ADD` then `SHR 1`.
- When `mid` is 0, `high = mid - 1` becomes -1. Use a *signed* comparison (`CMP low, high` then `JG`) on 64-bit registers so the loop exits correctly instead of wrapping around.
- Element access for a `dd` array: `LEA RSI, [ARR]` once, then `MOV EAX, [RSI+RBX*4]` (scale 4). For `db` arrays no scale is needed.
- Compare as signed or unsigned according to the data: `JL/JG` for signed values, `JB/JA` for unsigned (characters and the unsigned byte array in Problem 5.4).
- For descending order (Problem 5.4), swap the direction of the two branches.

Functions with stack parameters (Sections 5 and 7)

- The caller pushes arguments (64-bit registers or memory; zero- or sign-extend 32-bit values first). `CALL` then pushes the return address.
- In the callee, `MOV RBP, RSP` makes the *last* pushed argument `[RBP+8]`, the one before it `[RBP+16]`, and so on.
- End with `ret n`, where $n = 8\times$ (number of arguments), so `RSP` returns to its original value. For three arguments, `ret 24`.
- If a function changes registers the caller still needs, `PUSH` them at entry and `POP` them (in reverse order) before returning, and adjust the offsets accordingly.

General tips

- Writing a 32-bit register (e.g., `EAX`) zeroes the upper 32 bits of the 64-bit register; writing `AL` or `AX` does not.
- Before `DIV`, zero `RDX`. Before `IDIV`, sign-extend `RAX` into `RDX:RAX` (`CQO`, or set `RDX` to 0 or -1 by hand).
- `LOOP` uses `RCX` as a counter and does not check for 0 first; guard with `JRCXZ` when the count could be 0.
- Test every program with at least one "edge" input: the first element, the last element, a missing value, and an empty or one-element case.